

Manual de Comandos do NexusDB

Guia introdutório com exemplos comerciais sociais e científicos

Público Estudantes, pessoas iniciantes em bancos de grafos e equipes que desejam experimentar o NexusDB.

Escopo Inicialização, autenticação, bancos, nós, relacionamentos, consultas, índices, algoritmos de grafos, alta disponibilidade e grafos executáveis.

Versão do conteúdo Baseado no código e no README do projeto em 4 de setembro de 2026.

Aviso de uso O NexusDB é experimental. Use os exemplos para estudo, demonstrações e testes controlados. A camada HA não oferece todas as garantias de um consenso Raft completo.

Como usar este manual

Cada capítulo começa com uma ideia simples, mostra o comando completo e termina com uma aplicação próxima da realidade. Os exemplos usam o terminal do Windows e o comando curl. Troque SENHA_ADMIN pela senha criada na primeira inicialização.

Sumário

- 1 Conceitos de banco em grafo
- 2 Preparação e inicialização
- 3 Autenticação e formato das requisições
- 4 Bancos usuários e permissões
- 5 Nós e propriedades
- 6 Relacionamentos e consultas Cypher
- 7 Índices estatísticas e análise de consultas
- 8 Algoritmos de grafos e GraphRAG
- 9 Alta disponibilidade e recuperação
- 10 Grafos executáveis
- 11 Projetos guiados
- 12 Erros comuns glossário e referência rápida

Convenções dos exemplos

Texto	Significado
SENHA_ADMIN	Substitua pela senha real. Não inclua aspas extras no valor.
X-User	Usuário enviado no cabeçalho HTTP.
X-Pass	Senha enviada no cabeçalho HTTP.
X-Database	Banco lógico que receberá a operação.
127.0.0.1:7474	Endereço padrão de um servidor local.

1 Conceitos de banco em grafo

Um banco em grafo representa objetos como nós e ligações como relacionamentos. Esse modelo é útil quando as conexões importam tanto quanto os dados de cada objeto.

Nós propriedades e rótulos

Um nó representa algo identificável. O rótulo informa a categoria e as propriedades guardam seus atributos. Em uma loja, um nó Produto pode ter nome, preço e estoque. Em uma rede social, um nó Pessoa pode ter nome e cidade. Em um laboratório, um nó Amostra pode ter espécie, local e data.

Manual NexusDB

Adriano Lima e Souza – adrianoucam@gmail.com
Phd Student – UFF – Niterói – Brasil

Área	Nó	Propriedades possíveis
Comércio	Produto	sku, nome, preco, estoque
Rede social	Pessoa	nome, cidade, interesses
Ciência	Amostra	codigo, especie, temperatura

Relacionamentos

Um relacionamento liga dois nós e possui um tipo. A direção ajuda a ler o significado. Cliente COMPRA Produto, Pessoa SEGUE Pessoa e Pesquisador ANALISA Amostra são exemplos diretos.

```
(Cliente)-[:COMPRA]->(Produto)
(Pessoa)-[:SEGUE]->(Pessoa)
(Pesquisador)-[:ANALISA]->(Amostra)
```

Quando o grafo ajuda

- Recomendar produtos observando compras e interesses conectados.
- Encontrar comunidades e pessoas influentes em uma rede social.
- Rastrear quais amostras, métodos e pesquisadores produziram um resultado.

2 Preparação e inicialização

O NexusDB é escrito em Rust. Os scripts do Windows preparam builds debug e release e iniciam o servidor por meio de arquivos de configuração.

Definir a senha corretamente

No PowerShell, use a variável de ambiente abaixo. Na primeira inicialização, o servidor grava apenas o hash Argon2id no banco de configuração.

```
$env:NEXUSDB_ADMIN_PASSWORD = "uma-senha-longa-unica-e-segura"
```

No Prompt de Comando, não coloque espaços ao redor do sinal de igual. O formato set NOME = valor cria uma variável com nome incorreto.

```
set "NEXUSDB_ADMIN_PASSWORD=uma-senha-longa-unica-e-segura"
```

Compilar

```
.\build_once_windows.bat debug
.\build_once_windows.bat release
```

Perfil	Uso recomendado	Executável
debug	Desenvolvimento e diagnóstico	target\debug\nexusdb.exe
release	Medição de desempenho e uso prolongado	target\release\nexusdb.exe

Iniciar um servidor único

```
.\run_node1_single_for_load.bat
```

A forma geral do executável é nexusdb run --config arquivo. O script local usa um arquivo de configuração e verifica a existência do executável antes de iniciar.

```
.\target\release\nexusdb.exe run --config .\nexusdb_single.conf
```

Encerrar

Na janela do servidor, pressione Ctrl+C. O processo deixa de aceitar novas conexões, espera as requisições ativas e realiza flush e checkpoint. Para um serviço do Windows, use service stop.

```
.\target\release\nexusdb.exe service stop
.\kill_nexusdb_windows.bat
```

O parâmetro --force encerra processos à força e pode interromper trabalho em andamento. Reserve-o para quando o encerramento gracioso falhar.

3 Autenticação e formato das requisições

A API HTTP recebe credenciais e o banco alvo em cabeçalhos. O corpo é JSON. A maioria das respostas também é JSON e contém status, resultados e erros.

Modelo de uma chamada curl

```
curl -X POST http://127.0.0.1:7474/db/data/node/create ^
-H "Content-Type: application/json" ^
-H "X-User: admin" ^
-H "X-Pass: SENHA_ADMIN" ^
-H "X-Database: loja" ^
-d '{"label": "Produto", "properties": {"sku": "P-100", "nome": "Caderno",
"preco": 18.90}}'
```

Cabeçalhos

Cabeçalho	Função	Exemplo
Content-Type	Informa que o corpo usa JSON	application/json
X-User	Identifica a conta	admin
X-Pass	Autentica a conta	SENHA_ADMIN
X-Database	Seleciona o banco	loja

Ler a resposta

status igual a success indica conclusão. partial_error significa que parte de uma requisição em lote falhou; examine errors mesmo quando o HTTP retornar 200. status igual a error indica falha total. Em HA, uma escrita enviada a um follower pode ser rejeitada; envie-a ao leader.

HTTP	Interpretação comum
200	A rota respondeu; confirme o campo status.
400	JSON ou parâmetro inválido.
401	Credenciais inválidas.
403	Usuário autenticado sem permissão.
503	Nó HA ainda sincronizando ou indisponível.

4 Bancos usuários e permissões

Separe aplicações e equipes em bancos lógicos. Crie contas próprias e conceda somente os acessos necessários.

Criar e listar bancos

```
curl -X POST http://127.0.0.1:7474/db/admin/database/create ^
-H "Content-Type: application/json" -H "X-User: admin" -H "X-Pass: SENHA_ADMIN" ^
-d "{\"database\":\"loja\"}"

curl -X POST http://127.0.0.1:7474/db/data/cypher ^
-H "Content-Type: application/json" -H "X-User: admin" -H "X-Pass: SENHA_ADMIN" ^
-H "X-Database: loja" -d "{\"statements\":[{\"query\":\"SHOW DATABASES\"}]}"
```

Criar usuário e conceder acesso

```
curl -X POST http://127.0.0.1:7474/db/admin/user/create ^
-H "Content-Type: application/json" -H "X-User: admin" -H "X-Pass: SENHA_ADMIN" ^
-d "{\"username\":\"analista_loja\",\"password\":\"senha-inicial-com-12-chars\",\"role\":\"user\"}"

curl -X POST http://127.0.0.1:7474/db/admin/grant ^
-H "Content-Type: application/json" -H "X-User: admin" -H "X-Pass: SENHA_ADMIN" ^
-d "{\"username\":\"analista_loja\",\"database\":\"loja\",\"access\":[\"GRAFO_READ\"]}"
```

Use GRAFO_READ para consulta e GRAFO_WRITE para alteração. Operações destrutivas exigem ADMIN. Conceda escrita somente a processos que realmente criam ou alteram dados.

Trocar senha sem reiniciar

```
curl -X POST http://127.0.0.1:7474/db/auth/password/change ^
-H "Content-Type: application/json" -H "X-User: analista_loja" -H "X-Pass: SENHA_ATUAL" ^
-d "{\"new_password\":\"uma-nova-senha-longa-e-unica\"}"
```

Comandos administrativos Cypher

Comando	Efeito
SHOW DATABASES	Lista os bancos carregados.
SHOW LABELS	Lista rótulos e quantidade de nós no banco atual.
TRUNCATE Produto	Remove o conteúdo do rótulo Produto.
DROP LABEL Produto	Remove o rótulo e seus dados.
TRUNCATE DATABASE loja	Esvazia o banco loja.
DROP DATABASE loja	Remove o banco; system não pode ser removido.

5 Nós e propriedades

Nós guardam entidades do domínio. Comece com propriedades pequenas e estáveis, usando identificadores como sku, username ou codigo_amostra.

Criar um nó

```
curl -X POST http://127.0.0.1:7474/db/data/node/create ^
-H "Content-Type: application/json" -H "X-User: admin" -H "X-Pass: SENHA_ADMIN" ^
-H "X-Database: loja" ^
-d '{"label\\":\\"Produto\\",\\"properties\\":{\\"sku\\":\\"P-100\\",\\"nome\\":\\"Caderno\\",\\"preco\\":18.90,\\"estoque\\":40}}'
```

Criar vários nós

```
curl -X POST http://127.0.0.1:7474/db/data/node/bulk ^
-H "Content-Type: application/json" -H "X-User: admin" -H "X-Pass: SENHA_ADMIN" ^
-H "X-Database: loja" ^
-d '{"rows\\": [{\\"key\\":\\"p101\\",\\"label\\":\\"Produto\\",\\"properties\\":{\\"sku\\":\\"P-101\\",\\"nome\\":\\"Caneta\\",\\"preco\\":4.50}}, {\\"key\\":\\"p102\\",\\"label\\":\\"Produto\\",\\"properties\\":{\\"sku\\":\\"P-102\\",\\"nome\\":\\"Mochila\\",\\"preco\\":129.00}}]}'
```

A chave key ajuda o cliente a relacionar cada resposta com sua linha original. Em lotes, sempre verifique status e errors.

Pesquisar

```
curl -X POST http://127.0.0.1:7474/db/data/node/search ^
-H "Content-Type: application/json" -H "X-User: admin" -H "X-Pass: SENHA_ADMIN" ^
-H "X-Database: loja" ^
-d '{"label\\":\\"Produto\\",\\"filters\\":{\\"preco\\":{\\"$lte\\":20}},\\"order_by\\":\\"preco\\",\\"limit\\":10}'
```

Operador	Pergunta respondida	Exemplo
\$contains	O texto contém um trecho	{"nome":{"\$contains":"cad"}}
\$ne	O valor é diferente	{"cidade":{"\$ne":"Recife"}}
\$gt e \$gte	Maior que ou maior ou igual	{"estoque":{"\$gt":0}}
\$lt e \$lte	Menor que ou menor ou igual	{"preco":{"\$lte":20}}

Atualizar remover e desatachar

As rotas node/update, node/delete e node/detach recebem o id do nó. DETACH remove os relacionamentos ligados ao nó; DETACH DELETE remove as relações e o próprio nó. Confirme o id retornado pela criação ou busca antes de apagar.

6 Relacionamentos e consultas Cypher

A API Cypher recebe uma lista de statements. O NexusDB implementa um subconjunto da linguagem, por isso exemplos de outros bancos podem exigir adaptação.

Criar nós com Cypher

```
curl -X POST http://127.0.0.1:7474/db/data/cypher ^
-H "Content-Type: application/json" -H "X-User: admin" -H "X-Pass: SENHA_ADMIN" ^
-H "X-Database: social" ^
-d '{"statements":[{"query":"CREATE (p:Pessoa {nome:\"Ana\", cidade:\"Recife\"})\"},
{"query":"CREATE (p:Pessoa {nome:\"Bruno\", cidade:\"Olinda\"})\"}]}'
```

Criar relacionamento por ids

```
curl -X POST http://127.0.0.1:7474/db/data/cypher ^
-H "Content-Type: application/json" -H "X-User: admin" -H "X-Pass: SENHA_ADMIN" ^
-H "X-Database: social" ^
-d '{"statements":[{"query":"CREATE (a {id: 1})-[:SEGUE]->(b {id: 2})\"}]}'
```

Os números 1 e 2 são ids internos. Consulte os nós primeiro e substitua pelos ids reais da sua base.

Consultar nós e relações

```
MATCH (p:Pessoa) RETURN p LIMIT 20
MATCH (p:Pessoa {cidade:"Recife"}) RETURN p
MATCH (a:Pessoa)-[r:SEGUE]->(b:Pessoa) RETURN a, r, b
MATCH p=(a)-[*1..3]->(b) RETURN p LIMIT 50
```

A última consulta percorre caminhos de um a três saltos. Limites pequenos evitam respostas muito grandes em redes densas.

Saída em tabela

```
curl -X POST http://127.0.0.1:7474/db/data/table ^
-H "Content-Type: application/json" -H "X-User: admin" -H "X-Pass: SENHA_ADMIN" ^
-H "X-Database: loja" ^
-d '{"query":"SELECT sku,nome,preco FROM Produto LIMIT 20 OFFSET 0","columns":["sku",
"nome","preco"]}'
```

A rota table aceita SELECT simples por rótulo ou uma consulta MATCH e devolve linhas e colunas adequadas para telas administrativas e relatórios básicos.

7 Índices estatísticas e análise de consultas

Índices aceleram filtros repetidos em propriedades. Estatísticas e planos de consulta ajudam a descobrir onde a aplicação gasta tempo.

Criar índice

```
curl -X POST http://127.0.0.1:7474/db/index/create ^
-H "Content-Type: application/json" -H "X-User: admin" -H "X-Pass: SENHA_ADMIN" ^
-H "X-Database: loja" -d '{"label": "Produto", "field": "sku"}'

CREATE INDEX ON :Produto(sku)
```

Crie índices para propriedades usadas com frequência em filtros e junções lógicas, como sku, username e codigo_amostra. Índices demais ocupam espaço e aumentam o custo de escrita.

Estatísticas e esquema

```
CALL db.stats()
CALL db.schema.visualization()
CALL apoc.meta.stats()
```

Explicar uma consulta

```
curl -X POST http://127.0.0.1:7474/db/query/explain ^
-H "Content-Type: application/json" -H "X-User: admin" -H "X-Pass: SENHA_ADMIN" ^
-H "X-Database: social" ^
-d '{"query": "MATCH (p:Pessoa {cidade: \"Recife\"}) RETURN p\", \"profile\": \"BALANCED\"}'
```

Use EXPLAIN antes de consultas amplas. Uma consulta MATCH sem filtro seletivo pode varrer muitos nós. Adicione um filtro, um índice ou um LIMIT quando fizer sentido.

Manutenção

Rota ou comando	Finalidade
POST /db/admin/flush	Força persistência pendente.
POST /db/admin/compact	Compacta o banco indicado por X-Database.
GET ou POST /db/cache/stats	Exibe estatísticas do cache para admin.
POST /db/admin/backup	Cria backup conforme parâmetros da rota.
POST /db/admin/restore	Restaura backup; valide em ambiente separado.

8 Algoritmos de grafos e GraphRAG

Os comandos CALL executam análises nativas sobre o grafo. Eles ajudam a responder perguntas de caminho, influência, comunidade e estrutura.

Comando	Pergunta simples	Aplicação
CALL algo.degree()	Quem possui mais conexões	Perfis muito conectados
CALL algo.shortestPath(1,10)	Qual é o caminho mais curto	Rota entre entidades
CALL algo.dijkstra(1,10,"peso")	Qual caminho tem menor custo	Logística ou distância
CALL algo.pageRank(20)	Quais nós recebem ligações importantes	Influência social
CALL algo.connectedComponents()	Quais grupos estão desconectados	Coortes científicas
CALL graph.louvain(20)	Quais comunidades aparecem	Segmentos de usuários
CALL graph.statistics()	Qual é o tamanho e estrutura geral	Diagnóstico do conjunto

Exemplo social

Uma plataforma educacional pode executar PageRank para encontrar contas centrais na circulação de materiais. Centralidade não significa qualidade; o resultado precisa de interpretação humana e regras contra abuso.

```
CALL algo.pageRank(20) LIMIT 25
CALL graph.LabelPropagation(20) LIMIT 100
```

Exemplo científico

Em uma rede de colaboração, componentes conectados mostram grupos que nunca colaboraram entre si. O menor caminho pode revelar intermediários entre dois laboratórios.

```
CALL algo.connectedComponents()
CALL algo.shortestPath(12, 87)
```

Contexto para modelos de linguagem

GraphRAG coleta uma vizinhança do grafo para formar contexto. Ele não invoca um modelo automaticamente; a aplicação deve enviar o contexto ao provedor de LLM escolhido.

```
CALL graphRAG.context(1, 2) LIMIT 50
CALL graphRAG.prompt(1, 2) LIMIT 50
```

9 Alta disponibilidade e recuperação

No modo HA, três nós formam um laboratório de replicação. Dois membros constituem quorum. Somente o leader deve aceitar escritas.

Iniciar o cluster local

```
set "NEXUSDB_ADMIN_PASSWORD=uma-senha-longa-unica-e-segura"
set "NEXUSDB_HA_SECRET=segredo-aleatorio-com-pelo-menos-32-caracteres"
run_cluster_ha.bat
```

Todos os nós devem receber exatamente o mesmo segredo. Use NEXUSDB_HA_CLUSTER_AUTH=true, NEXUSDB_HA_STRICT=true e NEXUSDB_HA_ALLOW_SINGLE_NODE_LEADER=false.

Consultar o leader

```
curl http://127.0.0.1:7474/cluster/health
curl http://127.0.0.1:7475/cluster/health
curl http://127.0.0.1:7476/cluster/health
```

Procure role igual a LEADER e ready igual a true. O endpoint detalhado /cluster/leader exige credenciais administrativas.

```
curl -H "X-User: admin" -H "X-Pass: SENHA_ADMIN" http://127.0.0.1:7474/cluster/leader
```

Falha e retorno de um nó

- Se o leader cair e os dois followers continuarem conectados, eles podem eleger um novo leader.
- Um único nó isolado não tem quorum e deve rejeitar escritas.
- Ao reiniciar, um nó volta indisponível, encontra o leader e informa seu last_log_index.
- O nó solicita /cluster/sync, aplica entradas WAL ausentes e instala snapshot quando o WAL não cobre a lacuna.
- Somente depois do catch-up ele apresenta ready true como follower atualizado.

Nós em máquinas diferentes

Os peers podem usar endereços de máquinas distintas. Cada servidor mantém armazenamento local. Restrinja a porta à rede privada, sincronize os relógios e use VPN ou proxy TLS. O HMAC autentica mensagens do cluster, mas não cifra o tráfego.

```
NEXUSDB_MODE=HA
NEXUSDB_HA_SERVER_ID=1
NEXUSDB_HTTP_ADDR=0.0.0.0:7474
NEXUSDB_HA_PEERS=http://192.168.1.22:7474,http://192.168.1.23:7474
NEXUSDB_HA_CLUSTER_AUTH=true
NEXUSDB_HA_STRICT=true
NEXUSDB_HA_ALLOW_SINGLE_NODE_LEADER=false
NEXUSDB_ALLOW_INSECURE_REMOTE=true
```

NEXUSDB_ALLOW_INSECURE_REMOTE apenas confirma que o operador entende a exposição. Essa opção não protege a conexão. Nunca exponha a porta diretamente à internet.

10 Grafos executáveis

O módulo Rust `nexusdb::execution` representa workflows como grafos. Ele calcula nós prontos, executa tarefas independentes em ondas paralelas, valida resultados e registra evidências.

Tipos de nó

Tipo	Responsabilidade	Exemplo
Splitter	Divide uma entrada em subtarefas	Separar pedidos por região
Worker	Executa um agente ou LLM registrado	Classificar mensagens
Code	Executa lógica ou script registrado	Calcular imposto
Gate	Valida uma condição determinística	Conferir JSON ou testes
Human	Pausa para decisão humana	Aprovar reembolso alto

Arestas e condições

Aresta	Uso
dependency	O destino aguarda a conclusão exigida pela origem.
correction	Encaminha uma falha para uma tarefa corretiva; ciclos desse tipo são permitidos.
learning	Registra o caminho usado para derivar uma restrição aprendida.
approval	Libera uma etapa após aprovação.

As condições podem verificar status do nó de origem, existência de um valor ou igualdade em um caminho JSON Pointer da saída. Ciclos de dependency são rejeitados porque impediriam o cálculo de `ready_nodes`.

Gates evidências e retries

- `ExitCode` compara o código retornado por um processo com o valor esperado.
- `JSON Schema` básico confere a forma do objeto de saída.
- Testes aprovados exigem que a execução registre sucesso nos testes.
- `HumanApproval` aguarda uma decisão explícita de uma pessoa.
- Cada tentativa registra status, motivo e detalhes de evidência.
- Após falha, `retry` e sucesso, o grafo pode derivar uma restrição para evitar a repetição do erro.

Exemplo conceitual comercial

```
Receber pedido [Splitter]
-> verificar estoque [Code]
-> gerar mensagem ao cliente [Worker]
-> validar JSON obrigatório [Gate]
-> aprovar desconto acima do limite [Human]
-> registrar resultado [Code]
```

A aplicação registra executores por nome em ExecutorRegistry. O NexusDB não habilita chamadas de LLM nem scripts arbitrários sozinho. Essa separação permite que a equipe escolha provedores, limites e isolamento.

Fachada Rust

```
db.put_execution_graph(graph)?;
let saved = db.get_execution_graph("pedido-42")?;
let report = db.run_execution_graph("pedido-42", &registry)?;
db.approve_execution_node("pedido-42", "aprovacao", true, Some("supervisor"))?;
```

Três camadas

Camada	Conteúdo	Exemplo científico
Data	Dados brutos	Leitura de temperatura
Knowledge	Conceitos e relações interpretadas	Amostra exposta a calor
Execution	Quem produziu, validou e aprovou	Script, gate e pesquisador

11 Projetos guiados

Os três exercícios seguintes usam poucos tipos de nó e relações. Eles podem ser ampliados conforme o estudante domina consultas, índices e análise de grafos.

Projeto de catálogo comercial

Objetivo: cadastrar produtos e clientes, registrar compras e localizar produtos baratos em estoque.

- Crie o banco loja_escola.
- Cadastre três Produto e dois Cliente.
- Crie relações COMPRA entre clientes e produtos usando os ids retornados.
- Crie um índice em Produto(sku).
- Pesquise Produto com estoque maior que zero e preço menor ou igual a 50.

Projeto de rede social escolar

Objetivo: representar estudantes que seguem colegas e participam de grupos de estudo.

- Crie Pessoa e Grupo com nomes fictícios.
- Use SEGUE entre pessoas e PARTICIPA entre pessoa e grupo.
- Execute degree para observar os nós mais conectados.
- Execute connectedComponents para verificar se há grupos isolados.
- Explique por que conexão alta não prova influência positiva.

Projeto de proveniência científica

Objetivo: rastrear como uma medição se tornou um resultado aprovado.

- Na camada Data, crie Amostra e Medicao.
- Na camada Knowledge, relacione a medição a uma hipótese ou conceito.
- Na camada Execution, modele Code para análise, Gate para validar o esquema e Human para revisão.
- Force uma falha de validação, corrija a saída e observe as evidências de cada tentativa.
- Registre uma restrição aprendida somente quando a sequência falha, retry e sucesso estiver comprovada.

12 Erros comuns glossário e referência rápida

A maior parte dos problemas iniciais vem de variáveis mal definidas, credenciais ausentes, envio ao follower ou uso de uma sintaxe fora do subconjunto implementado.

Diagnóstico rápido

Sintoma	Causa provável	Ação
Senha não encontrada	Variável definida com espaços no CMD	Use set "NEXUSDB_ADMIN_PASSWORD=val or".
401	Usuário ou senha incorretos	Confira X-User e X-Pass.
403	Permissão insuficiente	Conceda GRAFO_READ, GRAFO_WRITE ou ADMIN conforme a tarefa.
FOLLOWER SLAVE	Escrita enviada ao nó errado	Consulte /cluster/health e envie ao leader.
503 SYNCING	Nó recuperando WAL ou snapshot	Aguarde ready true.
partial_error	Parte do lote falhou	Leia errors e corrija somente as linhas inválidas.
Query não suportada	Sintaxe fora do subconjunto	Use os padrões deste manual e valide por EXPLAIN.

Glossário

Termo	Explicação
Nó	Entidade armazenada no grafo.
Rótulo	Categoria de um nó, como Produto ou Pessoa.
Propriedade	Par chave e valor pertencente a um nó.
Relacionamento	Ligação tipada e direcionada entre dois nós.
Cypher	Linguagem de consulta de grafos; o NexusDB implementa um subconjunto.

Termo	Explicação
Leader	Membro HA que aceita e coordena escritas.
Follower	Membro HA que replica o estado do leader.
Quorum	Maioria necessária para decisões do cluster.
WAL	Registro sequencial de alterações usado no catch-up.
Snapshot	Cópia completa usada quando o WAL não cobre a lacuna.
Gate	Validador determinístico dentro de um grafo executável.
Evidência	Registro estruturado do resultado de uma tentativa.

Mapa de rotas

Tarefa	Rota principal
Criar banco	POST /db/admin/database/create
Criar usuário	POST /db/admin/user/create
Conceder acesso	POST /db/admin/grant
Criar nó	POST /db/data/node/create
Criar nós em lote	POST /db/data/node/bulk
Pesquisar nós	POST /db/data/node/search
Criar ou consultar relações	POST /db/data/cypher
Exibir tabela	POST /db/data/table
Criar índice	POST /db/index/create
Explicar consulta	POST /db/query/explain
Ver saúde HA	GET /cluster/health

Tarefa	Rota principal
Ver leader	GET /cluster/leader

Checklist antes de um teste

- O executável existe no perfil escolhido.
- A senha administrativa foi definida sem espaços indevidos.
- O servidor respondeu em /cluster/health ou na porta configurada.
- O banco de destino existe e X-Database está correto.
- O usuário possui a permissão necessária.
- Em HA, o destino da escrita é o leader com ready true.
- A resposta foi verificada por status, errors e código HTTP.

Fim do manual